Static block programs :

```
//illegal forward reference
class Demo
{
   static
     {
          i = 100; //Initialization is possible due to prepare phase
          IO.println(i); //Reading the value directly is not possible, Illegal Forward Reference
     }

   static int i;  //Post declaration
}

public class StaticBlockDemo5
{
  void main()
     {
       new Demo();
     }
}
```

Note : In post declaration we can perform write operation but directly read operation is not possible.

```
class Demo
{
   static
     {
          i = 100;
          //IO.println(i); //Illegal forward reference
          IO.println(Demo.i);    //Valid
   }

   static int i = 900;  //Post declaration
}

public class StaticBlockDemo6
{
   void main()
     {
          IO.println(Demo.i);
     }
}
```

```
class StaticBlockDemo7
{
     static
     {
     IO.println("Static Block");
       return;                          //error
     }

     void main()
     {
          IO.println("Main Method");
     }
}
```

```
class Test
{
     final static int A;  //static blank final field

     public static void initialize()
     {
          A = 100;
     }
}

public class StaticBlockDemo8
{
  void main()
     {
      Test.initialize();
     }
}
```

A static blank final field must be initialized inside staic block only.

```
class Test
{
   public static final Test t1  = new Test();  //t1 = null

     static
     {
       IO.println("static block");
     }

     {
       IO.println("Non static block");
     }

     Test()
     {
          IO.println("No Argument Constructor");
     }
}

public class StaticBlockDemo9
{
     void main()
     {
          new Test(); //Class Loading(Step 1) + Object creation(Step 2)
     }
}
```

Note : static block and static field both are having same priority in the class initialization phase so executed according to the order.

```
class Sample
{
     static
     {
          IO.println("Static Block");
          x = m1();
          IO.println("x from static block :"+Sample.x);
     }

     public static int m1()
     {
       IO.println("Static Method");
       return 100;
     }

     static int x = 900; //Post declration  //x = 900
}

public class StaticBlockDemo10
{
     void main()
     {
          IO.println("x from main method :"+Sample.x);
     }
}
```

```
class Demo
{
     public static void print()
     {
          x = 120;
          IO.println("x value is :"+x); //class name is not reqd  because static method is having low priority than
     }                                                                                             static block

     static int x;  //Post declaration
}

public class StaticBlockDemo11
{
     void main()
     {
          Demo.print();
     }
}
```

```
class Demo
{
     static
     {
      IO.println("Static Block");
   }

   static int a = printA();   //a = 10


   int b = printB();   //b = 20

   {
     IO.println("Instance Block");
   }

   Demo()
     {
     IO.println("Constructor");
   }

   static int printA()
     {
     IO.println("Static Variable");
     return 10;
   }

   int printB()
     {
     IO.println("Instance Variable");
     return 20;
   }
}
public class StaticBlockDemo12
{
     public static void main(String[] args)
     {
      new Demo(); //Class Loading + Object Creation
   }
}
```

Index No	Variables types	Declaration Position	Scope	Memory Area	Applicable Modifiers	Accessed by	Lifetime
01	Class Variable OR Static Field	Inside Class	Belongs to class	Method Area	private, protected, public, final, transient and volatile.	Class name OR Object reference	Entire program execution (class is unloaded)
02	Instance Variable OR Non-Static Field	Inside Class	Belongs to an object (non-static)	Heap Area	private, default, protected, public, final, transient and volatile.	Object reference	As long as object lives
03	Local Variable	Inside Method	Inside a method/block /constructor	Stack Area	Only final modifier is applicable.	Only inside method/block	Until method ends
04	Parameter Variable	Inside Method	Passed to methods/constructors	Stack Area	Only final modifier is applicable.	Passed when Calling method	Until method ends

- ➔ Invalid Modifier for **static field**: abstract, synchronized, native, strictfp and final + volatile.
- ➔ Invalid Modifier for **non-static field**: abstract, synchronized, native and strictfp.
- ➔ Invalid Modifier for **Local Variable**: Only final is allowed.
- ➔ Invalid Modifier for **Parameter Variable**: Only final is allowed.

Why we can't access non static field from a static context like static block OR static method OR static nested class directly

---

All the static members (static Field, static block, static method, static nested inner class) are loaded/executed at the time of **loading the .class file** into JVM Memory.

At class loading phase object is not created because object is created in the 2nd phase i.e. Runtime data area so at the TIME OF EXECUTION OF STATIC MEMEBERS i.e. CLASS LOADING PAHSE, NON STATIC VARIABLE WILL **NOT BE AVAILABLE** BY DEFAULT hence we can't access non static variable from static context[static block, static method and static nested inner class] without creating the object.

```
public class StaticDemo
{
     int x = 100; //layer2

     public static void main(String[] args) //Layer1
     {
          System.out.println("x value is :"+x);
     }
}

class Test
{
     int x;
     public Test(int x)
     {
          this.x = x;
     }

   public static void print() //layer1
     {
          IO.println(super.toString()); //Layer2 error
          IO.println(this.x); //Layer2 error
      IO.println(x); //Layer2 error
     }
}

public class StaticEx
{
     public static void main(String[] args)
     {
          Test t1 = new Test(9);
          Test.print();
     }
}
```